CIS 3300 · INHERITANCE AND POLYMORPHISM

Inheritance Basics

Part 1 of 4: the is-a relationship, extends, super, overriding, and protected

RELATIONSHIPS

Classes relate to each other in three ways

Dependence

`uses-a`

A Mechanic uses a Car when it inspects one

Earlier unit

Aggregation

`has-a`

A Garage has a list of Car objects

Earlier unit

Inheritance

`is-a`

A Manager is an Employee: a more specialized version

This unit

Say it out loud: “A manager is an employee” sounds right. “A garage is a car” does not.

VOCABULARY

A subclass is a more specialized version of its superclass



Superclass

Also called base class or parent class.
The more general class.

Subclass

Also called derived class or child class.
The more specialized class.

```
public class Manager extends Employee { ... }
```

A subclass usually holds more data than its superclass, not less. “Sub” comes from subset: every manager is in the set of employees.

Arrows point from the subclass to its superclass, the same direction extends points.

Java classes get exactly one direct superclass

Single



One parent

Classes: yes

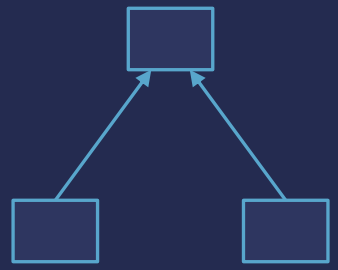
Multilevel



A chain of parents

Classes: yes

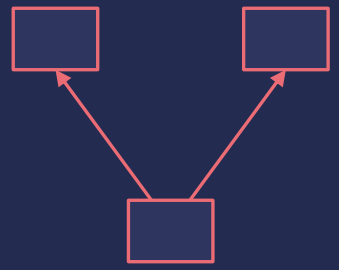
Hierarchical



Siblings share a parent

Classes: yes

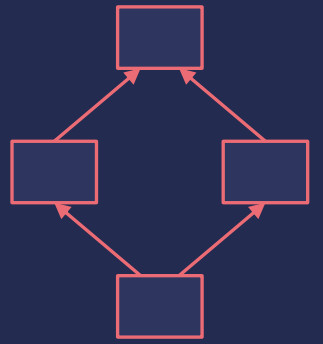
Multiple



Two or more parents

Interfaces only

Hybrid



A mix of the above

Interfaces only

Part 4 explains why multiple inheritance is limited to interfaces.

WHAT IS INHERITED

A subclass inherits state, behavior, and type, but not identity

State

Yes, as data

Every field is in the object, private ones too. Private fields are reached only through the superclass's methods.

Behavior

Accessible methods

public and protected methods, and package-private ones in the same package. Not private methods, not constructors.

Type

Yes

A Manager can go anywhere an Employee is expected, including every interface Employee implements.

Identity

No

Each object is its own object. Creating a Manager does not create a separate Employee.

In the earlier unit, an object had state, behavior, and identity. Inheritance passes on the first two, plus a type.

extends, super, and @Override work together

```
public class Manager extends Employee {
    private double bonus;

    public Manager(String name, double salary, double bonus) {
        super(name, salary);   // Employee part first
        this.bonus = bonus;
    }

    @Override
    public double getMonthlyPay() {
        return super.getMonthlyPay() + bonus / 12;
    }

    public double getBonus() { return this.bonus; }

    public void setBonus(double bonus) { this.bonus = bonus; }

}
```

Use as is

getId, getName, raiseSalary: never written in Manager, but every Manager has them.

Add

bonus, getBonus, setBonus exist only in Manager.

Replace

getMonthlyPay and toString are overridden; Manager's versions run.

annualSalary is private in Employee, so Manager asks super.getMonthlyPay() for that part of the answer.

TWO WORDS THAT SOUND ALIKE

Overriding replaces a method; overloading adds one

Overriding

- Same name and same parameter types, in a subclass
- Return type the same, or a subclass of it
- Access cannot get narrower
- Chosen at run time, by the object
- Always mark it with @Override

Overloading

- Same name, different parameter types or count
- In one class, or across a superclass and subclass
- Chosen at compile time, by declared types
- Example: println(int) and println(String)

@Override turns a typo like getMonthlypay() into a compiler error instead of a silent new method.

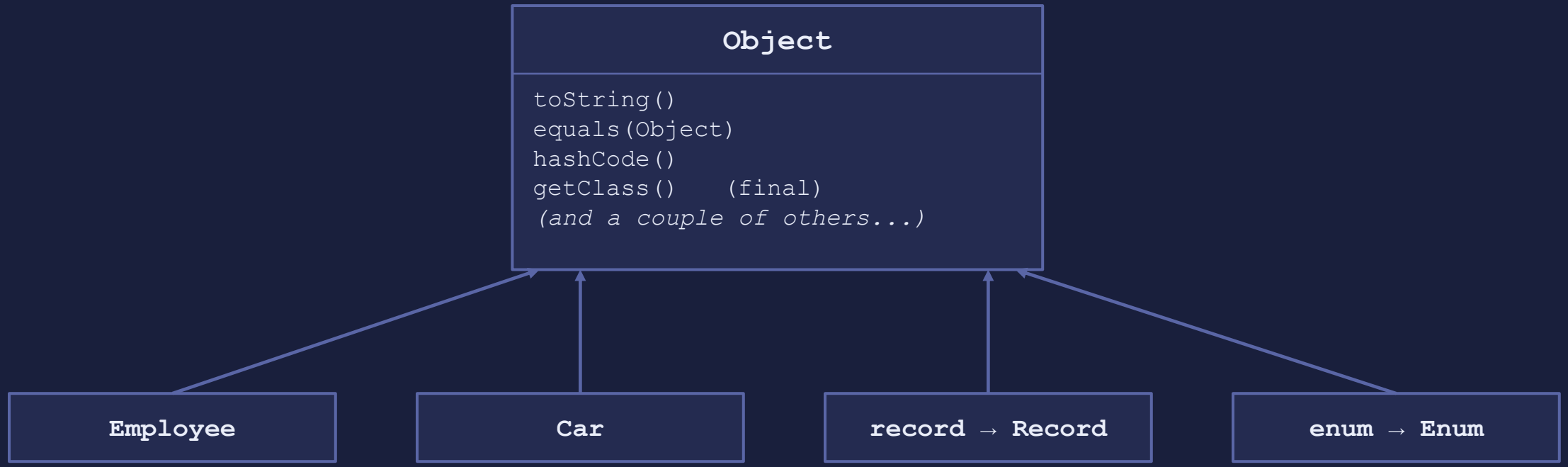
protected means my package, plus my subclasses anywhere

	Same class	Same package	Subclass, other package	Any other class
`public`	Yes	Yes	Yes	Yes
`protected`	Yes	Yes	Yes	No
`(none) package-private`	Yes	Yes	No	No
`private`	Yes	No	No	No

Keep fields private. Use protected for methods meant for subclasses. Leaving off a modifier is a real choice.

THE ROOT

Every class already extends Object



That is why toString, equals, and hashCode needed @Override in the earlier unit. Records and enums already use their one superclass slot by extending java.lang.Record and java.lang.Enum, so they cannot extend anything else.

NEXT

Next: build it in Example 1

1. Write Employee, then Manager extends Employee.
2. Remove super(...) and read the error. Then put it back.
3. Try to read annualSalary from Manager. Ask Employee instead.
4. Read the Part 1 notes and Core Java, Volume I, Section 5.1.

Watch for the surprise in Grace’s monthly pay. Part 2 explains it.